VEXEN

The permission layer for AI agents.

ALLOW · BLOCK · ASK A HUMAN

Agents now hold real credentials — they buy, refund, pay out and write to production. Their limits live in a prompt, which is a suggestion, not a permission. Vexen is the external check that runs **before** the action.

DETERMINISTIC ENGINE | APPROVALS THAT CANNOT BE REPLAYED | HASH-VERIFIED AUDIT | FAIL-CLOSED SDK

STAGE	USERS	ENGAGEMENT	TEAM	RAISING
Live, pre-revenue	5,000+ in 2 months	300+ daily active	Solo founder	$750K pre-seed

We gave AI hands and forgot the keys.

In the last eighteen months agents stopped suggesting and started acting. Tool use, MCP, computer use, agentic checkout. The same model that wrote a bad paragraph last year can now move money this year — and the blast radius moved with it.

COMMERCE

Agents that buy

Purchasing, restocking, ad spend, vendor payments — with a live card on file.

MONEY MOVEMENT

Agents that refund and pay out

Support refunds, credits, payouts, invoice approvals, reconciliation.

SYSTEMS OF RECORD

Agents that write to production

Customer records, pricing, inventory, emails sent on the company's behalf.

Capability arrived. The control layer did not.

A prompt is not a permission.

Today an agent's limits are a sentence someone typed into a system prompt. Nothing outside the agent verifies it. Nothing halts the action. Nothing proves what happened.

```
# the entire control system, today
SYSTEM_PROMPT = """
  You are a purchasing agent.
  Never spend more than $100.
  Always ask before refunding.
"""
```

FAILURE 1

No external check

The actor enforces its own limit. One prompt injection, one bad tool response, one hallucinated total — and the constraint evaporates silently.

FAILURE 2

No approval that holds

Hand-rolled approvals are replayable and amount-escalatable. A $10 approval gets reused for $10,000 — worse than no approval, because a signed-looking record now claims a human agreed.

FAILURE 3

No answer to “who allowed this?”

Evidence is scattered across model traces, vendor dashboards and application logs. Reconstruction takes hours, and nothing is tamper-evident.

Every team picks one of two bad options.

OPTION A

Keep the agent in read-only demo mode

Human confirms everything, or the agent never touches a real system. Safe, and it forfeits the entire reason the agent was built. The automation ROI never arrives — expensive, quietly, forever.

OPTION B

Hand it a real key and hope

Full credentials, prompt-level limits, no external stop. A risk that cannot be capped, priced, or explained to an auditor. Expensive once — and unexplainable afterwards.

> The blocker on agents in production is no longer capability. It is **authority** — and authority is an infrastructure problem, not a model problem.

Every computing generation grows a permission layer. This one hasn't.

MAINFRAME	NETWORK	CLOUD	SAAS IDENTITY	AUTONOMOUS AGENTS
RACF	Active Directory	IAM	Okta	Nothing

CAPABILITY

Tool use, MCP and computer use shipped in ~18 months. Agent count per company is rising monthly; each one needs credentials.

ACCOUNTABILITY ARRIVING

EU AI Act duties on human oversight and traceability. Auditors and insurers now ask "who approved this action?" — and there is no artefact to hand them.

THE WINDOW

Conventions for agent authority are being set right now, by whoever ships first. Identity took one cycle to standardise and never re-opened.

You cannot verify intent. You can only constrain consequence.

We never solved this for people by making people more trustworthy. We solved it with external limits: spending authority, dual control, segregation of duties, audit trails. A completely honest bank teller still cannot move ten million dollars alone.

WHERE THE INDUSTRY IS LOOKING

Make the model more trustworthy

Alignment, better prompts, evals, guardrails, content filters. All useful. None of it stops an action, and all of it puts a probabilistic system in charge of its own limits.

WHERE THE ANSWER IS

Make the authority external and provable

Deterministic permissions the agent does not control, human approval on consequential actions, and a tamper-evident record. Boring infrastructure, which is exactly why it works.

> The actor does not get to decide the limits of its own authority.

One call. Three answers.

One decorator above the function that performs the side effect. Vexen decides before anything happens — and if Vexen is unreachable, the function does not run.

- ALLOW
- APPROVAL REQUIRED
- BLOCK

FAIL-CLOSED SDK

NO AGENT-SIDE TRUST

REASON CODE + TRACE

```
@vexen.guard("purchase.create",
             amount=lambda o: o.total)
def place_order(order):
    # money actually moves here
    stripe.PaymentIntent.create(...)

# agent calls it
place_order(order)
# -> ALLOW          runs once
# -> APPROVAL_REQ   halts, phone buzzes
# -> BLOCK          never runs
```

Deterministic by design. No model in the enforcement path.

A 13-step, default-deny evaluation with no I/O and no inference. The same request always returns the same answer, and the trace is the explanation.

01 – 03	04 – 06	07 – 09	10 – 12	13
Identify agent, workspace, action	Status, expiry, permission grant	Limits, resource and action scope	Org rules, approval state, claim	Emit decision, reason code, trace

DEFAULT DENY

Anything not explicitly granted is refused. New capability is never silently permitted.

AI WRITES RULES, NEVER ENFORCES

Plain English becomes a schema-validated rule, shown back before it takes effect. The model never touches the decision.

EXPLAINABLE BY CONSTRUCTION

Every answer carries a machine-readable reason code and the full evaluation path.

Approvals that actually hold.

This is where most implementations quietly fail. A half-correct approval is worse than none, because a signed-looking record then claims a human agreed.

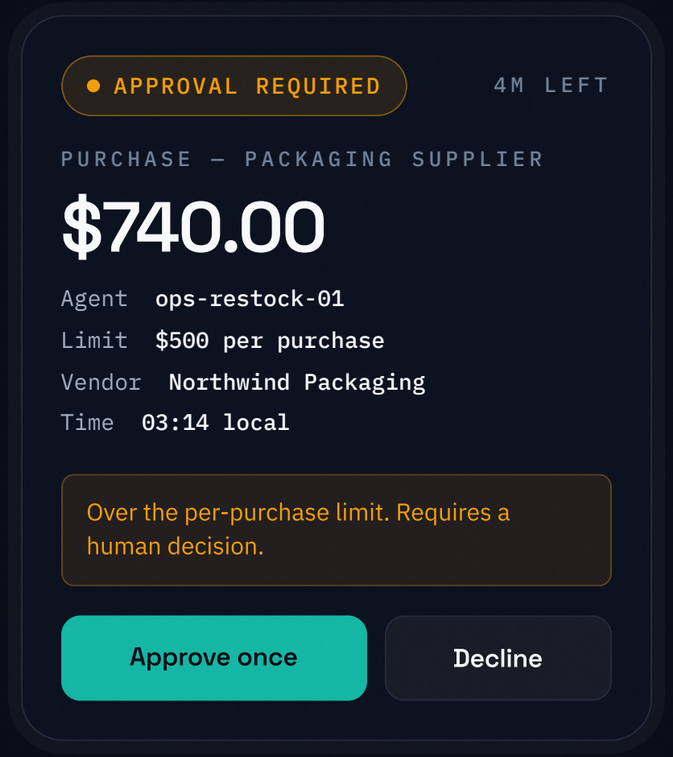
INVARIANT	WHAT VEXEN DOES	THE ATTACK IT KILLS
Single use	Consumed on first use, atomically claimed at the database layer	`Replay the same approval twice`
Time bound	Hard TTL; expiry is evaluated, not assumed	`Approval reused days later`
Concurrency safe	Claim is atomic, never read-then-write	`Two parallel calls both spend one approval`
Scope bound	Locked to permission, resource ID and amount to the cent	`$10 approval used for $10,000`
Scan safe	Per-recipient signed links; the decision is POST-only	`Corporate link scanner approves a $40,000 purchase`
Fail closed	If Vexen is unreachable the guarded function does not run	`Outage becomes an open door`

Replay, amount escalation, concurrent claim and cross-tenant access are covered tests — not intentions.

Two taps. No login. Fifteen seconds.

The person accountable for the money is rarely the person who deployed the agent. If approving is hard, they will disable the control — so approval had to be easier than turning it off.

- **No account needed.** Signed, per-recipient link. Nothing to install, nothing to remember.
- **Full context.** Which agent, what action, exact amount, why it paused.
- **Exactly once.** Approve and the action completes a single time, then the token is dead.
- **Email and web push.** Reaches a lock screen, not an inbox nobody reads.



A record you can hand to someone who does not trust you.

Every decision, grant, rule change and approval is written to a hash-verified log with an integrity pass that reports exactly which rows were altered.

```
GET /api/audit/verify

{
  "rows_checked": 41208,
  "chain_valid":  true,
  "altered_rows": [],
  "algorithm":    "sha-256"
}
```

TAMPER EVIDENT

Change one historical row and verification names it. Deterrence, not just record-keeping.

ONE QUESTION, ONE PLACE

"Who allowed this?" resolves in seconds — agent, permission, amount, approver, reason, timestamp — instead of hours across three systems.

BUILT FOR THE BUYER

The artefact an auditor, insurer or regulator will accept. This is the piece that turns a developer tool into a purchase order.

Governed in under a minute, by someone who has never seen an API key.

Security products die from configuration burden. Nobody should design a permission posture from a blank page.

10,639
Ready-made agent configurations — 55 archetypes across 28 industries

67
Business-language permissions across 18 categories, with limits and expiry

4
Server-side roles enforced across 18 individually guarded endpoints

1
Line of code to enforce — one decorator above the function that acts

“Any purchase over $100 needs a person to approve it.” Typed in plain English, converted into a schema-validated rule, shown back for confirmation before it takes effect — and enforced deterministically, with no model in the path.

$740 of packaging, at three in the morning.

A restocking agent finds a supplier, builds a basket and calls the purchase function. Its limit is $500. Nobody is awake.

Step 1
Agent calls the guarded function with a $740 total

Step 2
Vexen returns APPROVAL_REQUIRED — over the per-purchase limit

Step 3
Execution halts. No charge, no partial state, no retry loop

Step 4
Ops lead's phone buzzes; two taps at 7am approve it

Step 5
Purchase completes exactly once; replay is refused; record is hashed

WITHOUT VEXEN

Either the agent never had the authority to help at 3am, or it spent $740 and someone found out from a bank alert.

WITH VEXEN

The agent kept its usefulness, the company kept its limit, and there is a record naming who allowed the exception.

Built solo in two months. Live, and past five thousand users.

No paid acquisition, no outbound, no funding. Content-led, the same way my previous five products reached 100,000+ users between them.

5,000+
Registered users, 2 months

2,000+
Monthly active — 40% of signups

800+
Weekly active

300+
Daily active — 15% DAU/MAU

10,000+
Actions completed in product

~12%
Month-over-month user growth

55/55
End-to-end regression checks passing

110/110
Independent backend pass, no critical defects

Pre-revenue by choice — free while the enforcement guarantees are finished.

Users walk themselves toward production traffic.

Observed 1	Observed 2	Observed 3	Observed 4	Outcome
Deploy a ready-made configuration	Write a rule in plain English and test it	Trigger an approval, approve it on a phone	Go looking for the SDK	Guarded calls in their own codebase

WHAT THEY ASK FOR, UNPROMPTED

- **TypeScript SDK** — Python-only is the current stopping point
- **MCP gateway and framework guards** — enforce where agents already run
- **“Can I stop the agent bypassing this at all?”** — a request for the credential broker

WHY THAT MATTERS

Nobody asks for a stricter constraint on themselves for a nice-to-have. Users are pulling the product toward its **more defensible** version — the one where enforcement is unavoidable rather than voluntary. That request is the roadmap, and it is the difference between a tool and infrastructure.

Priced per decision, not per seat.

Revenue grows with how much a customer automates, not with how many people they employ — one of the few software lines that expands as headcount falls.

TIER	PRICE	WHO AND WHAT TRIGGERS IT	GATED ON
Free	$0	Developer installs the SDK, monthly decision allowance	Nothing — this is the adoption path
Self-serve	$99 – $999 / mo	Crosses the allowance, upgrades without a conversation	Decision volume, seats, retention
Mid-market	$12K – $25K / yr	A second team adopts it; someone asks for roles and reporting	RBAC, reporting, support
Enterprise	$50K – $100K / yr	Security review of an agent going to production	SSO, SCIM, retention, pentest, SOC 2, credential broker

LAND

One engineer, one decorator, free tier.

EXPAND

More agents → more decisions → higher tier, with no new seats sold.

CONVERT

The security review is the pricing event. The broker and SOC 2 are what make it a line item.

The plan, and the number that decides whether it works.

MONTHS 0 – 6

$250K ARR

- Credential broker shipped
- TypeScript SDK, MCP gateway
- 10 design partners in production
- First 25 paying accounts

MONTHS 7 – 12

$1.2 – 1.5M

- Published external pentest
- SOC 2 Type I, SSO, SCIM
- First 10 enterprise contracts
- Founding engineer hired

MONTHS 13 – 18

$4M

- 3 platform embeds — frameworks and vertical AI vendors
- Open authority spec v0.1
- Logos growing faster than headcount

MONTHS 19 – 24

$10M

- 100 enterprise @ $60K = $6.0M
- 250 mid-market @ $14K = $3.5M
- 250 self-serve @ $2.4K = $0.6M

THE ONE METRIC THAT DECIDES IT

Decisions per account per month. Users grow because I write; decisions grow only if agents genuinely run behind it. Needs ~30–40% quarterly compounding — that is where expansion revenue comes from.

SAID PLAINLY

$10M in 24 months is the aggressive case. My base case is **$3–5M**, with $10M reachable only if the platform embeds land and carry distribution.

Bought once. Removed almost never.

Permission layers are the stickiest line in the stack, because removing one means choosing to run an ungoverned system. Nobody does that to save money.

- **Bottom-up SAM.** ~150K companies running action-taking agents by 2028 × ~$20K average annual spend ≈ **$3B**. A nearer-term 50K × $20K ≈ $1B.
- **Comparables.** IAM software is a $20B+ market; Okta alone is around $2.6B in revenue. Agent authority is the same shape of layer, one level down.
- **Better unit of growth.** Every company has more actions than employees, and the ratio is moving in one direction.

WHERE WE PLAY FIRST

Agents that touch money

E-commerce and retail ops, support refunds, finance payouts and invoices — 20 to 200 person companies with the exposure and no security team.

THEN

Platforms that resell governance

Agent frameworks and vertical AI vendors who must offer permissions and approvals to their own customers, and would rather not build it badly.

THEN

Regulated write paths

Financial services, healthcare and government — where the audit artefact is not a preference, it is the condition of deployment.

Four categories point at this problem. None of them stop anything.

APPROACH	BLOCKS THE ACTION	NON-REPLAYABLE HUMAN APPROVAL	TAMPER-EVIDENT AUDIT	USABLE BY A NON-ENGINEER	NO MODEL IN THE DECISION PATH
Hand-rolled if-statements (the real incumbent)	~	×	×	×	✓
Agent observability (LangSmith, Langfuse)	×	×	~	~	✓
Guardrail libraries (text filtering)	~	×	×	×	×
Policy engines (OPA, Cedar)	✓	×	×	×	✓
Framework human-in-the-loop	~	~	×	~	✓
Vexen	✓	✓	✓	✓	✓

CATEGORY IS UNOWNED

No standard vocabulary, no pricing norm, no analyst category. That is the opportunity and the risk.

WHO COULD ENTER

Okta or Auth0 from identity, Stripe from payments, or a framework bundling a “good enough” feature. Speed and correctness are the defence.

Four defences, in increasing order of durability.

1 — POSITION

On the write path, not the read path

Observability sits beside the action; Vexen sits in front of it. Once a guarded call is in production code, removing it is a decision to run ungoverned.

2 — CORRECTNESS

Invariants that take real adversarial work

Atomic single-use claims, cent-level scope binding, scan-safe POST-only actions, hash-verified audit, fail-closed defaults. Easy to demo, hard to actually get right.

3 — BOTH PERSONAS

The engineer integrates, the accountable human signs

Policy engines serve only developers, so nobody with a budget can use them. Plain-English rules and two-tap approvals put the buyer inside the product.

4 — STANDARD

The credential broker, then the artefact

Short-lived scoped credentials make bypass impossible. Then a portable, verifiable statement of bounded authority that travels between companies — the primitive agent commerce and agent insurance both need, and neither has.

From a check an agent chooses to make, to a constraint it cannot avoid.

WHEN	SHIP	WHY IT CHANGES THE BUSINESS
Now → M3	Credential broker — short-lived, scoped, single-purpose credentials per approved action	Enforcement stops being voluntary. This is what supports a $60K contract instead of a $12K one.
M2 → M5	TypeScript SDK, MCP gateway, native framework guards	Removes the top adoption blocker and puts enforcement where agents already run.
M4 → M9	Published external penetration test, SOC 2 Type I, SSO and SCIM	Unlocks the enterprise security review, which is the pricing event.
M6 → M12	10 design partners with agents in production, first paid contracts	Converts proven developer demand into a security budget line.
M12 → M18	Open specification v0.1 for agent authority, Vexen as reference implementation	Turns a product into a convention — the version an incumbent structurally cannot start.

> Today an agent can route around Vexen if its author wants to. The broker is first on the list for exactly that reason — and it is the objection I would rather name than argue with.

Sixth product. Over 100,000 users before this one.

Ashrit Talluri — solo founder. Every product below was designed, built, shipped and distributed by me, with no funding and no paid acquisition.

HealthLens	AI wellness platform unifying health data, food and voice analysis	70,000+
AI-Namo	Government document fraud verification — **adopted by a local government**	20,000+
GreenGuide	Agricultural copilot for disease and contamination risk	10,000+
CerebroAegis	Health and safety guidance in plain language	5,000+
Vexen	The permission layer for AI agents — live, two months old	5,000+

COMMERCIAL

COO of The1550Formula, an AI-first SAT prep company — ran operations to ~$70K/month with 1,000+ students improving ~200 points. Perfect 1600 SAT myself.

DISTRIBUTION

Content-led only. Educational channel past 3M views — which is how you reach the engineer the week before they ship an agent with a production key.

THE HONEST GAP

Solo, and I have never sold a security control through enterprise procurement. Hire 1: a founding engineer out of IAM, fraud or payments to own the broker. Hire 2: go-to-market.

The four things most likely to kill this.

RISK — NO FORCING FUNCTION

No job title owns “agent authority”. Engineers feel a permission layer as friction; the exposed executive does not know the exposure exists.

MITIGATION

Sell the incident, not the feature: show a buyer what their agents are currently permitted to do. The audit artefact is what an auditor, insurer or regulator asks for — and those three are arriving.

RISK — BYPASSABLE TODAY

The credential broker is designed, not built, so a rigorous buyer says this is a convention rather than a control and waits.

MITIGATION

It is the first thing this round funds, with a founding engineer out of IAM or payments hired to own it. Fair objection, dated answer.

RISK — INCUMBENT BUNDLING

Okta, Stripe or an agent framework ships a “good enough” governance feature.

MITIGATION

Bundled features stop at logging and pausing. Correct approval invariants, scan-safe delivery and tamper-evident audit are the hard 20% nobody bundles — and an open spec makes us the reference, not the competitor.

RISK — SOLO FOUNDER, PRE-REVENUE

Concentration risk, no enterprise sales experience, and a business that has never charged anyone.

MITIGATION

55% of this round is two hires that close exactly those gaps. Runway is not the constraint: $50K in the bank against ~$200/month burn.

Every gap here is one I would rather name than have found in diligence.

$750K pre-seed to make bypassing impossible.

Post-money SAFE, $8M cap, comfortable in a $7–10M range. No capital raised to date, no investors, no process run. 100% founder-owned, nothing promised to anyone.

Two hires — founding engineer, first GTM 55%

IAM / fraud / payments engineer to own the credential broker; GTM to run ten design partners.

Founder full-time and infrastructure 25%

The first product I want to be the only thing I do.

Security and compliance 20%

Published external pentest, SOC 2 Type I, SSO and SCIM — the enterprise gate.

WHAT IT BUYS, IN MILESTONES

- Credential broker in production
- TypeScript SDK and MCP gateway shipped
- 10 design partners with agents live
- First paid contracts, ~$1M ARR at M12
- Open authority spec v0.1 published

JUDGE IT YOURSELF

Block a real agent and confirm the side effect never happened. Try to replay an approval, escalate its amount, or alter an audit row. Hand the console to a non-engineer. Take the service offline and watch the guarded call refuse to run.

Agents already have the keys.
Somebody has to hold the lock.

LIVE PRODUCT

5,000+ users, 300+ daily active

BUILT BY

One person, two months

NEXT

The credential broker

RAISING

$750K pre-seed